

Entity Framework Context Class in C# with Examples

In this article, I am going to discuss the **Entity Framework Context Class** in C# with Examples. Please read our previous article where we discussed the [Architecture of Entity Framework](#) in Detail. At the end of this article, you will understand what exactly the Context Class is and when and how to use this Context Class in Entity Framework with Examples.

What is Entity Framework Context Class in C#?

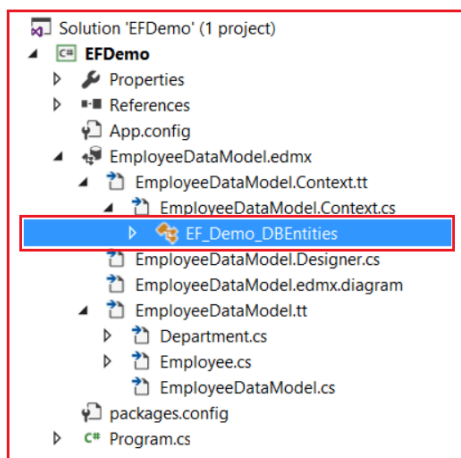
The Entity Framework allows us to Query, Insert, Update, and Delete data using Common Language Runtime (CLR) objects which are also known as Entities (or .NET Classes). The Entity Framework maps the entities (classes) and relationships that are defined in our model to a database.

The primary class that is responsible for interacting with the database is `System.Data.Entity.DbContext`. The Context class in Entity Framework is a class that derives from `DbContext` in EF 6. It is an important class in Entity Framework, which represents a session with the underlying database. The `DbContext` class belongs to `System.Data.Entity` namespace.

According to MSDN, A `DbContext` instance represents a combination of the Unit Of Work and Repository patterns such that it can be used to query from a database and group together changes that will then be written back to the store as a unit. `DbContext` is conceptually similar to `ObjectContext`.

We are going to work with the same example that we created in our [Introduction to Entity Framework](#) article. In that console application, we retrieve the data from the SQL Server database using Entity Framework.

First, let's have a look at the solution. Please double-click on the **EF_Demo_DBEntities** which is inside the **EmployeeDataModel.Context.cs** which is inside **EmployeeDataModel.Context.tt** file as shown below.



The following **EF_Demo_DBEntities** class is an example of a context class that is created by Entity Framework using C# Language.

```
//-----  
// <auto-generated>  
// This code was generated from a template.  
//  
// Manual changes to this file may cause unexpected behavior in your application.  
// Manual changes to this file will be overwritten if the code is regenerated.  
// </auto-generated>  
//-----  
namespace EFDemo  
{  
    using System;  
    using System.Data.Entity;  
    using System.Data.Entity.Infrastructure;  
  
    public partial class EF_Demo_DBEntities : DbContext  
    {  
        public EF_Demo_DBEntities()  
            : base("name=EF_Demo_DBEntities")  
        {  
        }  
  
        protected override void OnModelCreating(DbModelBuilder modelBuilder)  
        {  
            throw new UnintentionalCodeFirstException();  
        }  
  
        public virtual DbSet<Department> Departments { get; set; }  
        public virtual DbSet<Employee> Employees { get; set; }  
    }  
}
```

In the above code, as you can see the **EF_Demo_DBEntities** class is derived from **DbContext** class which makes it a context class. It also includes an entity set for **Departments** and **Employees** entities. The context class is used to query or save data to the database i.e. using this context class we can perform the **SELECT, INSERT, UPDATE, and DELETE** database operations on the database tables.

Further, if you notice this context class has one parameterless constructor and this constructor is calling the **DbContext** class parameterized constructor by passing **EF_Demo_DBEntities** as the value for the Name parameter. Here, you need to pass either the database name or the connection string name. Here, we are passing the connection string name. If you remember while adding the ADO.NET Entity DataModel, we specified the connection string name as **EF_Demo_DBEntities**, and Entity Framework will create the connection string in the App.Config file. Now, if you go to the App.Config file, then you will see that the connection string created with the name **EF_Demo_DBEntities** is as follows:

```
<connectionStrings>
<add
connectionString="metadata=res://*/EmployeeDataModel.csdl|res://*/EmployeeDataModel.ssdl|res://*/EmployeeDataModel.msl;provider=System.Data.SqlClient;provider
connection string=&quot;data source=LAPTOP-ICA2LCQL\SQLEXPRESS;initial catalog=EF_Demo_DB;integrated
security=True;MultipleActiveResultSets=True;App=EntityFramework&quot;; providerName=System.Data.EntityClient" />
</connectionStrings>
```

So, when we are going to perform the CRUD operation using the above Entity Framework context class, it is going to use the above connection string and going to perform the database operation on the database.

Note: If we are using Entity Framework in Desktop Applications like Console, and Windows Form, then Entity Framework will add the connection string in App.Config file and if we are using Entity Framework in a Web Application like Web Form, MVC, and Web API then Entity Framework will add the connection string in Web.Config file.

How to Query using Entity Framework Context Class in C#?

In order to understand how to query using the Entity Framework Context class, please modify the Program class as shown below. The following example code is self-explained, so please go through the comment lines.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace EFDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities DBEntities = new EF_Demo_DBEntities())
            {
                //Fetch all the Departments data
                //Here, Departments is a property of EF_Demo_DBEntities context class
                //The Departments property mapped with the Departments table of the database
                //When we call the Departments property on the context object, it will return the
                //departments list from the Department database table
                List<Department> listDepartments = DBEntities.Departments.ToList();
                //Accessing all the elements of the listDepartments collection
                foreach (Department dept in listDepartments)
                {
                    Console.WriteLine($" Department = {dept.Name}, Location = {dept.Location}");
                    //There is a primary key and foreign key relationships between the Employee and Departments
                    //The Department entity contains a property called Employees which will retrieve all the
                    //employees of that department
                    foreach (Employee emp in dept.Employees)
                    {
                        Console.WriteLine($" \t Name = {emp.Name}, Email = {emp.Email}, Gender = {emp.Gender}, salary
= {emp.Salary}");
                    }
                    Console.WriteLine();
                }
                Console.ReadKey();
            }
        }
    }
}
```

Output:

```

Department = IT, Location = Mumbai
    Name = Mark, Email = Mark@g.com, Gender = Male, salary = 60000
    Name = Pam, Email = Pam@g.com, Gender = Female, salary = 60000
    Name = Ben, Email = Ben@g.com, Gender = Male, salary = 70000
    Name = John, Email = John@g.com, Gender = Male, salary = 80000

Department = HR, Location = Delhi
    Name = Philip, Email = Philip@g.com, Gender = Male, salary = 45000
    Name = Mary, Email = Mary@g.com, Gender = Female, salary = 30000

Department = Sales, Location = Hyderabad
    Name = Steve, Email = Steve@g.com, Gender = Male, salary = 45000
    Name = Sara, Email = Sara@g.com, Gender = Female, salary = 345000
    Name = Valarie, Email = Valarie@g.com, Gender = Female, salary = 35000

```

In the above example, I am showing how to query using the Entity Framework Context Class and in our upcoming articles, I will show you how to perform INSERT, UPDATE, and DELETE operations using Entity Framework Context Class. Now, if you want to check what SQL Command is generated by the Entity Framework, then you can take the help of the SQL Profiler tool using SSMS. The following SQL commands are generated by Entity Framework for the above example.

To Fetch all the Departments i.e. `DBEntities.Departments.ToList()`, the following script is generated by the Entity Framework

```

SELECT
[Extent1].[ID] AS [ID],
[Extent1].[Name] AS [Name],
[Extent1].[Location] AS [Location]
FROM [dbo].[Departments] AS [Extent1]

```

To Fetch all Employees of Each Department i.e `dept.Employees`, the following script is generated by the Entity Framework and the following SQL Statement will be executed for each department.

```

exec sp_executesql N'SELECT
[Extent1].[ID] AS [ID],
[Extent1].[Name] AS [Name],
[Extent1].[Email] AS [Email],
[Extent1].[Gender] AS [Gender],
[Extent1].[Salary] AS [Salary],
[Extent1].[DepartmentId] AS [DepartmentId]
FROM [dbo].[Employees] AS [Extent1]
WHERE [Extent1].[DepartmentId] = @EntityKeyValue1',N'@EntityKeyValue1 int',@EntityKeyValue1=1

```

Now, if you go to the definition of the Department Entity (class) which is auto-generated by the Entity Framework, then you will see that along with the ID, Name, and Location properties, it also has one property Employees which is a collection type as shown in the below image.

```

public partial class Department
{
    [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage", "CA2214:DoNotCallOverridableMethodsInConstructors")]
    public Department()
    {
        this.Employees = new HashSet<Employee>();
    }

    public int ID { get; set; }
    public string Name { get; set; }
    public string Location { get; set; }

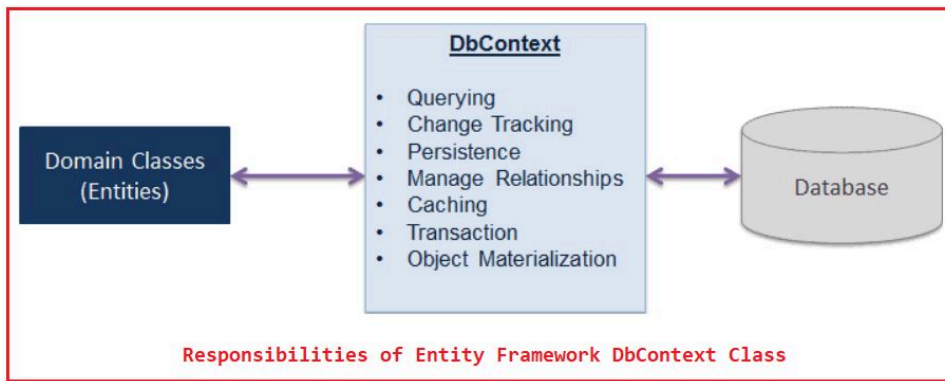
    [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage", "CA2227:CollectionPropertiesShouldBeReadOnly")]
    public virtual ICollection<Employee> Employees { get; set; }
}

```

This Property will return list of Employees of a Department

What are the Responsibilities of DbContext in Entity Framework?

The DbContext class is one of the important classes in Entity Framework. It is a bridge between your domain or entity classes and the underlying database. For a better understanding, please have a look at the below image.



So, the following are the major responsibilities performed by the DbContext class:

1. **Querying:** Converts LINQ-to-Entities queries to SQL queries and then sends those queries to the database for execution.
2. **Change Tracking:** DbContext class also keeps track of the changes that occurred on the entities after querying from the database. For example, after fetching the data from the database, you have updated a few data, insert new data and delete some existing data. This information will keep tracked by the DbContext class.
3. **Persisting Data:** The DbContext class is also responsible for performing the Insert, Update and Delete operations to the database, based on entity states.
4. **Caching:** DbContext class also Provides first-level caching by default. In our upcoming articles, we will discuss this concept in detail with Examples.
5. **Transaction:** It also provides the support to implement Transactions using the Unit of Work and Repository Pattern.
6. **Manage Relationship:** This class is also responsible for managing the relationships between the entities using CSDL, MSL, and SSDL in Db-First or Model-First approach, and using fluent API in the Code-First approach. We will discuss CSDL, MSL, SSDL, and Fluent API in detail in our coming articles.
7. **Object Materialization:** It is also used to convert the raw data which is retrieved from the database into entity objects.

In the next article, I am going to discuss the [Entities in the Entity Framework](#) in Detail with Examples. Here, in this article, I try to explain the **Context Class in Entity Framework** with Examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article